

Interactive 2D Heat Diffusion Simulation

Backend Task Division

Python + NumPy + SciPy

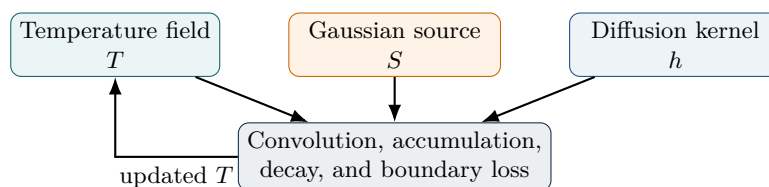
Scope. Backend and scientific computation only. This document deliberately excludes UI, web frameworks, routes, and mouse/drag handling. Candle position and intensity are ordinary function parameters; a future UI can simply call the functions again whenever those values change.

Contents

1	System at a Glance	1
2	Person 1 — Physical Model & Time Evolution	2
2.1	temperature_field.py	2
2.2	heat_source.py	2
2.3	diffusion_model.py	2
3	Person 2 — Convolution & Frequency-Domain Processing	3
3.1	kernel.py	3
3.2	spatial_convolution.py	4
3.3	frequency_domain.py	4
4	Integration Contract & Backend Pipeline	5
5	Core Deliverables	5
6	Learning Checklist	5

1 System at a Glance

The simulation represents temperature as a sampled two-dimensional field $T(x, y, t)$. At every time step, a smooth candle source is spatially spread by a diffusion kernel, added to the field, then reduced through cooling and boundary loss.



2 Person 1 — Physical Model & Time Evolution

Ownership: the temperature field, the candle's source model, and time evolution: accumulation, decay, and boundary loss.

2.1 temperature_field.py

This module contains the field itself and basic field-level operations. It does not perform time evolution; it only defines the data representation and simple access/reset behavior.

```
create_field(width: int, height: int, ambient_temp: float = 20.0) -> np.ndarray
```

Creates a two-dimensional temperature grid initialized entirely to `ambient_temp`. Returns a floating-point array of shape `(height, width)`.

```
reset_field(T: np.ndarray, ambient_temp: float = 20.0) -> np.ndarray
```

Resets every cell in `T` to the ambient temperature and returns an array with the same shape.

```
get_temperature_at(T: np.ndarray, x: int, y: int) -> float
```

Reads the temperature at one grid cell. This supports testing, debugging, and future hover-to-inspect behavior.

Math concept. At one fixed time, `T` is a discretized form of the continuous signal $T(x, y, t)$: each array cell is one sampled location in space.

2.2 heat_source.py

The candle is modeled as a finite spatial source, rather than as a single-pixel impulse. A two-dimensional Gaussian makes the source smooth, physically plausible, and friendly to FFT-based processing.

```
generate_gaussian_source(
    width: int, height: int, position: tuple[float, float],
    intensity: float, sigma: float
) -> np.ndarray
```

Builds the source signal $S(x, y)$ centered at `position = (x0, y0)`:

$$S(x, y) = intensity \exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2}\right).$$

The result has shape `(height, width)`. Setting intensity to zero naturally turns the candle off.

```
combine_sources(sources: list[np.ndarray]) -> np.ndarray
```

Extension. Sums multiple source arrays through linear superposition: the total effect of independent candles is the sum of their individual effects.

Math concept. A Gaussian has a center, amplitude, and spread (σ). Unlike an ideal impulse $\delta(x, y)$, it represents a finite, smooth physical heat source.

2.3 diffusion_model.py

This is the time-evolution engine. It combines Person 1's field and source with Person 2's convolution implementation.

```

apply_temporal_decay(
    T: np.ndarray, ambient_temp: float, decay_rate: float, dt: float
) -> np.ndarray

```

Pulls each cell toward ambient using Newton’s-law-of-cooling-style exponential decay:

$$T_{\text{new}} = T_{\text{ambient}} + (T - T_{\text{ambient}})e^{-\text{decay_rate } dt}.$$

```

apply_boundary_loss(T: np.ndarray, loss_rate: float) -> np.ndarray

```

Reduces temperature in edge cells to model heat leaving the simulated environment instead of reflecting back into it.

```

step(
    T: np.ndarray, S: np.ndarray, kernel: np.ndarray, decay_rate: float,
    boundary_loss_rate: float, dt: float, convolve_fn
) -> np.ndarray

```

Performs a single update:

1. Compute `spread = convolve_fn(S, kernel)`.
2. Accumulate: `T = T + spread`.
3. Apply temporal decay toward ambient.
4. Apply edge-cell boundary loss.

The callable `convolve_fn` is injected rather than hard-coded, so this module depends only on the convolution contract, not on how convolution is implemented.

```

run_simulation(
    T: np.ndarray, get_source_fn, kernel: np.ndarray, decay_rate: float,
    boundary_loss_rate: float, dt: float, num_steps: int, convolve_fn
) -> list[np.ndarray]

```

Calls `get_source_fn()` once per iteration, advances the field with `step(...)`, and returns the field at every time step for later plotting or animation.

```

set_diffusion_coefficient(D: float) -> float

```

Validates and returns D , rejecting negative values. This gives the backend one explicit owner for the adjustable diffusion coefficient before it is passed to the kernel generator.

Math concept. This module implements a discrete-time difference equation, $T[n + 1] = f(T[n])$. The diffusion coefficient controls spreading through the kernel built by Person 2.

3 Person 2 — Convolution & Frequency-Domain Processing

Ownership: the heat-spreading kernel, spatial convolution, FFT/IFFT operations, and the convolution-theorem implementation.

3.1 kernel.py

```

generate_diffusion_kernel(
    diffusion_coefficient: float, dt: float, size: int
) -> np.ndarray

```

Builds a `size x size` Gaussian kernel from the heat equation's fundamental solution:

$$h(x, y) = \frac{1}{4\pi Ddt} \exp\left(-\frac{x^2 + y^2}{4Ddt}\right).$$

Normalize the result so that it sums to 1, ensuring convolution redistributes heat instead of adding or removing it. Larger D produces a wider kernel and therefore farther spreading per time step.

Math concept. The Green's function of the heat equation is a Gaussian whose width grows over time. This directly links physical diffusion to convolution with a Gaussian kernel.

3.2 spatial_convolution.py

```
convolve_2d(field: np.ndarray, kernel: np.ndarray) -> np.ndarray
```

Computes 2D convolution and returns an array matching `field`'s shape. A suitable implementation is `scipy.signal.fftconvolve(field, kernel, mode='same')`; conceptually, this is spatial convolution: slide the kernel over the field and sum weighted overlaps.

3.3 frequency_domain.py

```
compute_fft2(field: np.ndarray) -> np.ndarray  
compute_ifft2(field_freq: np.ndarray) -> np.ndarray
```

The first function calculates the two-dimensional FFT with `np.fft.fft2`. The second returns `np.real(np.fft.ifft2(field_freq))`, because the physical temperature field is real-valued.

```
convolve_via_fft(S: np.ndarray, h: np.ndarray) -> np.ndarray
```

Implements the convolution theorem:

$$\text{conv}(S, h) = \text{IFFT}(\text{FFT}(S) \cdot \text{FFT}(h)).$$

The function must zero-pad `h` to `S`'s shape before transforming it, and returns a real array matching `S`. This is typically the function supplied to `step(...)` as `convolve_fn`.

```
get_power_spectrum(field: np.ndarray) -> np.ndarray
```

Computes the shifted power spectrum $|\text{FFT}(\text{field})|^2$ with `np.fft.fftshift`, placing the zero-frequency component in the center for frequency-domain visualization.

```
frequency_domain_filter(  
    field_freq: np.ndarray, filter_type: str, cutoff: float  
) -> np.ndarray
```

Extension. Attenuates or removes components above or below a cutoff for 'lowpass' and 'highpass' filtering. It can demonstrate, for example, that removing high spatial frequencies smooths the heatmap.

Math concepts. Two-dimensional DFTs, spatial-frequency content, the convolution theorem (convolution in space equals multiplication in frequency), zero-padding, and frequency-domain filtering.

4 Integration Contract & Backend Pipeline

The one interface between the two owners

(source_array, kernel_array) → spread_array

All arrays belong to the same shape family. `diffusion_model.step()` calls this function without needing to know whether it is implemented directly in space or through FFTs.

Person 1		Person 2
<code>create_field()</code>	→	temperature field T
<code>generate_gaussian_source()</code>	→	source array S
diffusion coefficient D	→	<code>generate_diffusion_kernel(D, dt)</code>
<code>step(T, S, kernel, ...)</code>	<u>calls</u> →	<code>convolve_via_fft(S, kernel)</code> or <code>convolve_2d(S, kernel)</code>
updated temperature field T	→	<code>get_power_spectrum(T)</code>

Data sent from Person 1 to Person 2 are the source array S and diffusion coefficient D . Data returned are the kernel h and the spread field `convolve_fn(S, h)`; Person 1 accumulates that result into the temperature field.

5 Core Deliverables

Function group	Owner	Status
<code>create_field, reset_field, get_temperature_at</code>	P1	Core
<code>generate_gaussian_source</code>	P1	Core
<code>combine_sources</code>	P1	Extension
<code>apply_temporal_decay, apply_boundary_loss, step, run_simulation</code>	P1	Core
<code>set_diffusion_coefficient</code>	P1	Core
<code>generate_diffusion_kernel</code>	P2	Core
<code>convolve_2d</code>	P2	Core
<code>compute_fft2, compute_ifft2, convolve_via_fft</code>	P2	Core
<code>get_power_spectrum</code>	P2	Core
<code>frequency_domain_filter</code>	P2	Extension

6 Learning Checklist

Person 1

1. NumPy arrays, indexing, slicing, and broadcasting.
2. Discretized 2D signals $T(x, y)$.
3. Gaussian center, amplitude, and spread.
4. Difference equations and exponential decay.
5. Boundary conditions and heat loss at edges.
6. Convolution as spreading a source with a kernel.
7. A simple time-stepping loop.

Person 2

1. NumPy arrays, indexing, slicing, and broadcasting.
2. 1D convolution intuition before 2D convolution.
3. 2D discrete convolution and `scipy.signal.fftconvolve`.
4. Fourier transforms and spatial frequency content.
5. `fft2`, `ifft2`, and `fftshift`.
6. The convolution theorem.
7. Zero-padding before FFT convolution.
8. Low-pass and high-pass filtering (extension).